

Ejercicio — Catálogo de libros

Contexto

Se necesita construir una API REST en **Node.js + TypeScript** para gestionar un catálogo de libros. La API debe organizarse en una **arquitectura en capas** y usar **Express** como framework HTTP y **SQLite** como base de datos.

Datos de cada libro

De cada libro interesa almacenar:

- Un identificador único (ISBN)
- El título
- El autor
- El género literario
- La cantidad de páginas (número entero positivo)

Nota: el autor se almacena como texto (solo su nombre). No es necesario modelarlo como una entidad separada ni armar relaciones en la base de datos.

Funcionalidades requeridas

Consultas

- **Listar el catálogo:** se debe poder obtener todos los libros disponibles. La lista debe ser paginable (indicando página y cantidad de resultados por página) y permitir filtrar por título o por ISBN de forma parcial.
- **Consultar un libro:** se debe poder obtener todos los datos de un libro específico a partir de su ISBN.

Modificaciones

- **Agregar un libro:** se debe poder incorporar un nuevo libro al catálogo indicando todos sus datos.

- **Reemplazar un libro:** se debe poder actualizar completamente los datos de un libro existente, enviando todos los campos.
 - **Modificar un libro:** se debe poder actualizar solo algunos campos de un libro sin necesidad de enviar el resto.
 - **Eliminar un libro:** se debe poder dar de baja un libro del catálogo.
-

Reglas de negocio

- No pueden existir dos libros con el mismo ISBN.
 - Al reemplazar un libro, todos los campos son obligatorios; si falta alguno, la operación debe rechazarse.
 - Al modificar parcialmente un libro, solo se actualizan los campos enviados; los demás permanecen sin cambios.
 - La paginación tiene como valor predeterminado la primera página con 10 resultados. El máximo de resultados por página es 100.
-

Navegabilidad de las respuestas (HATEOAS)

Cada respuesta debe incluir las **acciones disponibles** sobre el recurso devuelto, de forma que el cliente pueda descubrir qué operaciones puede realizar a continuación sin necesidad de conocer de antemano la estructura de la API.

Por ejemplo, al consultar un libro la respuesta debe indicar cómo actualizarlo, modificarlo parcialmente, eliminarlo y volver al listado. Al listar libros, la respuesta debe indicar cómo ir a la página siguiente, a la anterior y cómo crear un nuevo libro.

Escenarios que debe manejar el sistema

El sistema debe responder correctamente en los siguientes casos:

- Agregar un libro con todos sus datos → el libro queda guardado y se devuelven sus datos.
- Intentar agregar un libro con un ISBN que ya existe → se informa el conflicto.
- Intentar agregar un libro sin alguno de los campos requeridos → se informa el error.
- Consultar un libro existente → se devuelven sus datos.
- Consultar un libro que no existe → se informa que no fue encontrado.

- Listar el catálogo → se devuelve la lista paginada con los metadatos de navegación.
 - Listar filtrando por título → se devuelven solo los libros que coincidan.
 - Reemplazar completamente un libro existente → los datos quedan actualizados.
 - Intentar reemplazar un libro que no existe → se informa que no fue encontrado.
 - Modificar parcialmente un libro existente → solo los campos enviados se actualizan.
 - Eliminar un libro existente → el libro queda eliminado.
 - Intentar eliminar un libro que no existe → se informa que no fue encontrado.
-

Sugerencias para orientarse en la solución

El ejercicio puede abordarse de forma incremental, construyendo el sistema de a pasos. A continuación se describe una secuencia posible:

Paso 1 — Servidor básico

Empezar por lo mínimo: un servidor HTTP con Express que responda a una única ruta de prueba. El objetivo es entender la estructura necesaria para que el servidor funcione antes de agregar recursos o base de datos.

Paso 2 — Primeras consultas con datos en memoria

Modelar el recurso `Book` y exponer los dos endpoints de lectura (listar y consultar por ISBN). En este paso los datos pueden vivir en un array en memoria; todavía no hace falta base de datos. Esto permite enfocarse en cómo se construyen las rutas y cómo se extraen parámetros del request.

Paso 3 — CRUD completo en memoria

Completar los seis endpoints del CRUD usando los verbos HTTP con su semántica correcta (GET, POST, PUT, PATCH, DELETE). Agregar las validaciones básicas y los códigos de respuesta adecuados para cada situación. Los datos pueden seguir en memoria.

Paso 4 — Arquitectura en capas + SQLite

Refactorizar el código separando las responsabilidades en capas independientes y reemplazar el array en memoria por una base de datos SQLite real. La funcionalidad debe ser idéntica a la del paso anterior; lo que cambia es la organización del código y la persistencia de los datos.

Paso 5 — Capa de servicios y errores de dominio

Introducir una capa de servicios entre el controlador y el repositorio. La lógica de negocio (¿puede crearse este libro? ¿existe el que se quiere modificar?) debe migrar al servicio, expresando los problemas en términos del dominio (no de HTTP). El controlador queda con responsabilidades exclusivamente HTTP: parsear el request, delegar al servicio y traducir los errores de dominio en códigos de respuesta.

Paso 6 — Navegabilidad (HATEOAS)

Agregar a todas las respuestas los links hacia las acciones disponibles sobre el recurso. Para la colección, incluir también los links de paginación (página siguiente y anterior). Conviene aislar la construcción de estos links en un módulo propio para que pueda reutilizarse desde cualquier endpoint.